

IoT Alarm System

Engineering Report

Author: Filippos Kladouchas

Project Type: Embedded Systems / IoT

Programming Language: C++

Platform: NodeMCU (ESP8266)

Development Environment: Arduino IDE

1. Introduction

This project presents the design and implementation of a low-cost Internet of Things (IoT) alarm system developed around the NodeMCU ESP8266 microcontroller. The system combines conventional alarm components with embedded control and cloud connectivity, providing both local and remote monitoring and control capabilities.

The system was designed as a complete functional prototype rather than as an isolated software or electronics demonstration. It integrates alarm sensors, audible warning devices, a local keypad interface, power regulation and signal-isolation circuitry, embedded firmware, cloud services, remote control, and automated notifications into a unified system.

The local alarm subsystem is responsible for sensor monitoring, user authentication, alarm state management, and siren control. Remote functionality is provided through integration with the Arduino IoT Cloud, allowing system variables to be monitored and controlled through a configurable mobile dashboard. Cloud triggers are additionally used to generate notifications following relevant security events.

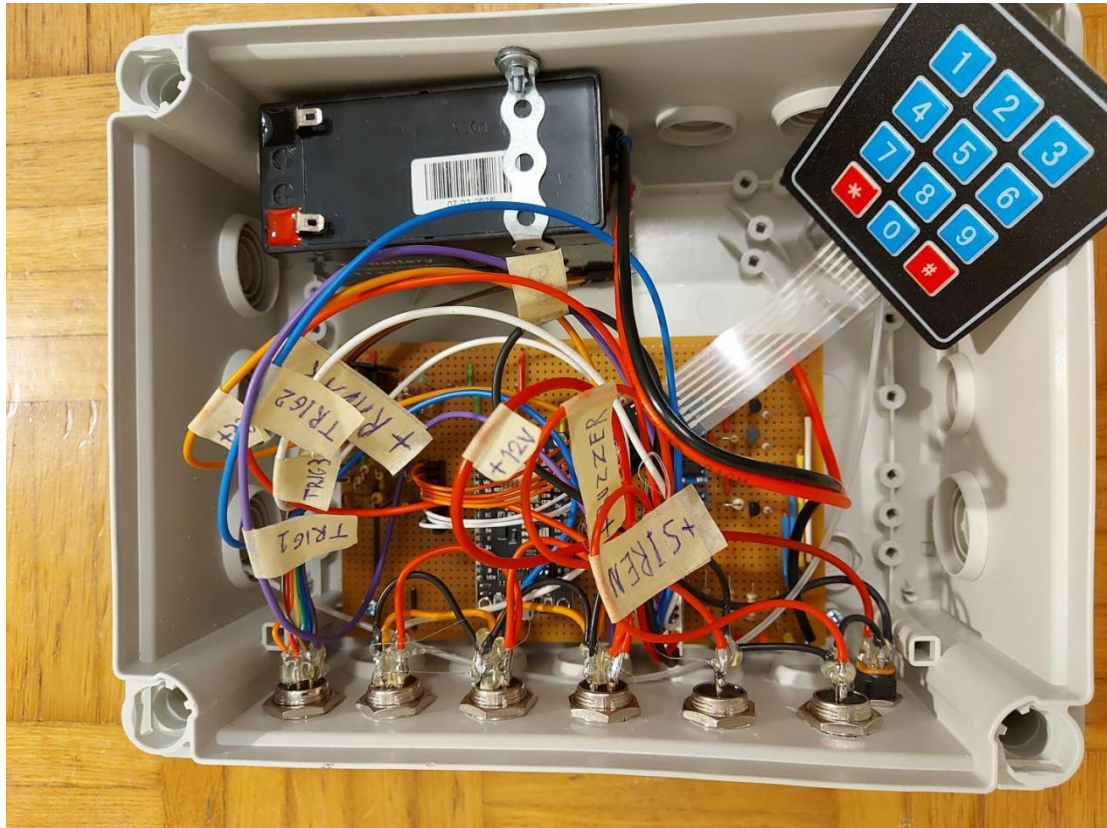
A primary objective of the project was to investigate whether a functional IoT-enabled security system could be implemented using inexpensive and widely available components while maintaining a modular architecture that could be extended with additional sensors and functionality. The implemented prototype was therefore designed around commonly available 12 V alarm components and the Wi-Fi-enabled NodeMCU ESP8266.

Particular attention was given to the interaction between the low-voltage microcontroller and the external alarm hardware. The implementation includes dedicated voltage regulation, optocoupler-based electrical isolation, transistor-controlled outputs, and an analog voltage-divider interface that allows a standard 3×4 matrix keypad to be read through a single analog input.

On the software side, the embedded firmware was structured using an object-oriented architecture, with separate components responsible for sensor management, keypad operation, siren control, and overall system coordination. State-based control logic manages the progression from normal monitoring through detection, notification, pre-alarm, and alarm conditions.

The resulting prototype demonstrates the integration of embedded systems, electronic circuit design, IoT connectivity, and cloud-based monitoring within a single low-cost security platform.

Figure 1 – Implemented IoT alarm system prototype



2. Design Objectives and Engineering Approach

The IoT Alarm System was developed with the objective of combining the core functionality of a conventional alarm installation with remote monitoring and control through IoT technologies. Rather than following a fixed reference architecture, the implementation was designed around a set of practical engineering objectives concerning cost, functionality, expandability and integration with conventional alarm components.

A primary objective was to maintain the fundamental alarm functionality at the embedded-system level. Sensor monitoring, local user input, alarm-state management and siren control are handled by the microcontroller, while cloud connectivity extends the system with remote monitoring, control and notification capabilities.

Cost and component availability were also important considerations during the design process. The prototype was therefore constructed using inexpensive, commercially available components while retaining compatibility with common 12 V alarm peripherals. The resulting implementation demonstrates that IoT functionality can be integrated into a conventional alarm architecture without requiring specialized or expensive hardware.

Several engineering decisions were made during development to accommodate the characteristics and limitations of the selected platform. Dedicated voltage regulation was implemented for the low-voltage controller, while optocouplers and transistor stages were used to interface it with the external 12 V circuitry. Similarly, rather than dedicating seven

digital I/O connections to the 3×4 keypad, a resistor network was designed to encode individual key presses as distinct analog voltage levels, allowing the complete keypad to be monitored through a single analog input.

Modularity was also considered in the firmware architecture. Separate software components were developed for sensor management, keypad operation, siren control and overall system coordination. This separation allowed the behavior of individual subsystems to remain independently manageable while being integrated into the overall alarm-control logic.

The system was ultimately implemented as a complete physical prototype, allowing the interaction between embedded firmware, electronic circuitry, alarm peripherals and cloud services to be evaluated as an integrated system.

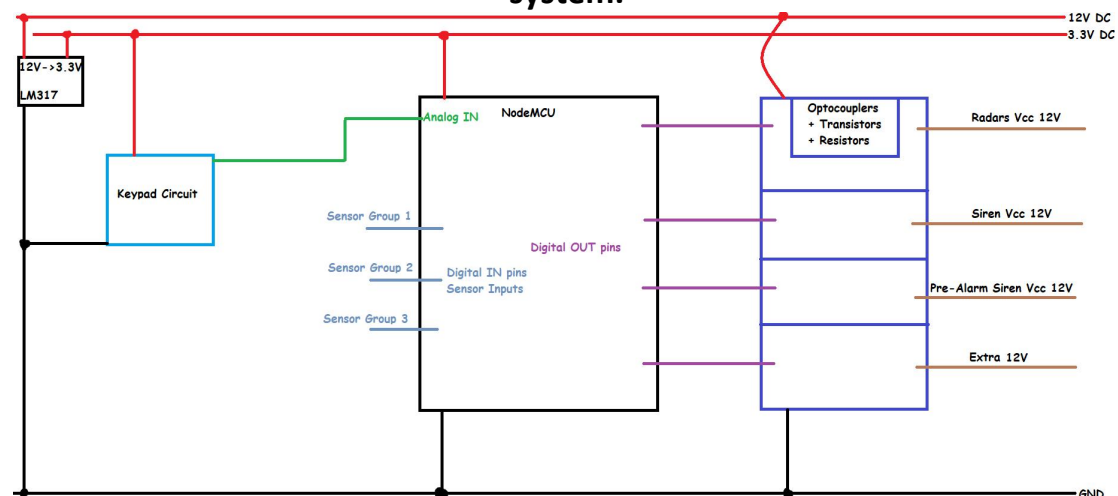
3. System Architecture

The IoT Alarm System was designed as an integrated embedded and cloud-connected platform in which local alarm functionality is combined with remote monitoring, control and notification services. The overall architecture can be divided into four main subsystems: the local alarm system, the cloud platform, the remote user interface, and the notification system.

At the center of the local system is the ESP8266-based controller, which coordinates the operation of the connected sensors, keypad and alarm outputs. The controller continuously evaluates the state of the system and executes the corresponding alarm logic locally. This includes processing sensor activity, interpreting keypad input, managing operating states and controlling the sirens and other outputs.

The hardware interfaces with conventional 12 V alarm components while the microcontroller operates at a lower voltage. Power regulation, signal isolation and transistor-based output stages therefore form an intermediate hardware layer between the controller and the external alarm devices. The analog keypad interface is also integrated into this layer, converting individual key presses into voltage levels that can be interpreted through a single analog input.

Figure 2. General circuit architecture of the implemented alarm system.



Remote functionality is provided through the Arduino IoT Cloud. Selected system variables are synchronized between the embedded controller and the cloud platform, allowing the current state of the alarm and its associated sensors to be remotely observed and, where appropriate, modified. A customized dashboard created using the available Arduino IoT Cloud interface elements provides the user-facing controls and status indicators.

The cloud platform also provides the connection between the embedded system and the remote notification functionality. Cloud triggers can react to changes in relevant system variables and generate notifications following predefined events. This allows remote event reporting to complement the locally executed alarm logic without transferring the fundamental control of the alarm system away from the embedded controller.

From a software perspective, the local firmware is divided into dedicated components responsible for the microcontroller coordination logic, sensors, sirens and keypad. These components interact through the main control logic, which manages the overall operating state of the system and coordinates responses to local and remote events.

The resulting architecture separates the system into distinct functional layers while maintaining communication between them:

Physical environment → Sensors / Keypad → Embedded Controller → Alarm Outputs

Embedded Controller ↔ Arduino IoT Cloud ↔ Remote User Interface / Notifications

This separation allows the local alarm functionality and the IoT functionality to coexist within the same system while retaining clearly defined responsibilities for sensing, control, actuation and remote interaction.

4. Hardware Design

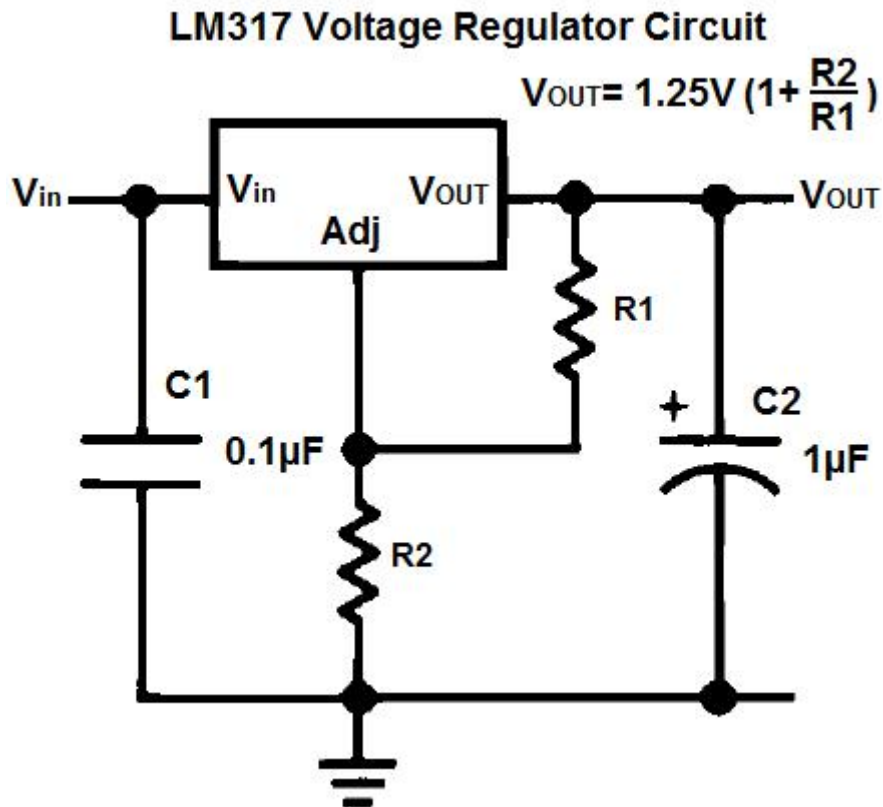
The hardware subsystem was designed to integrate the ESP8266-based controller with conventional alarm components operating primarily at 12 V. Since the controller and the external alarm circuitry operate at different voltage and current levels, the implementation incorporates dedicated power regulation, electrical isolation and output-driving stages.

4.1 Power Supply and Voltage Regulation

The alarm peripherals and backup battery operate around a 12 V supply, while the microcontroller requires a substantially lower operating voltage. An LM317 adjustable voltage regulator was therefore incorporated into the design to derive the controller supply from the main power source.

The regulator was configured using a 220 Ω resistor for R1 and a combined resistance of 352 Ω for R2, together with 0.1 μF and 1 μF capacitors for stabilization. During implementation, the regulator output was measured at approximately 3.33 V.

Figure 3. LM317 voltage-regulation circuit.



This approach allowed the control electronics and the 12 V alarm infrastructure to operate from the same overall power system while maintaining the voltage level required by the controller.

4.2 Electrical Isolation and Output Control

Direct connection between the low-voltage controller and the external alarm circuitry was avoided through the use of optocouplers. These provide electrical isolation between the controller side and the 12 V side of the system.

Transistor switching stages were used where the external loads required voltage or current levels that could not be supplied directly by the microcontroller outputs. This arrangement allowed the controller to activate alarm peripherals while remaining electrically separated from the higher-voltage circuitry.

The implemented architecture therefore follows the general signal path:

Microcontroller output → Optocoupler → Transistor stage → 12 V peripheral

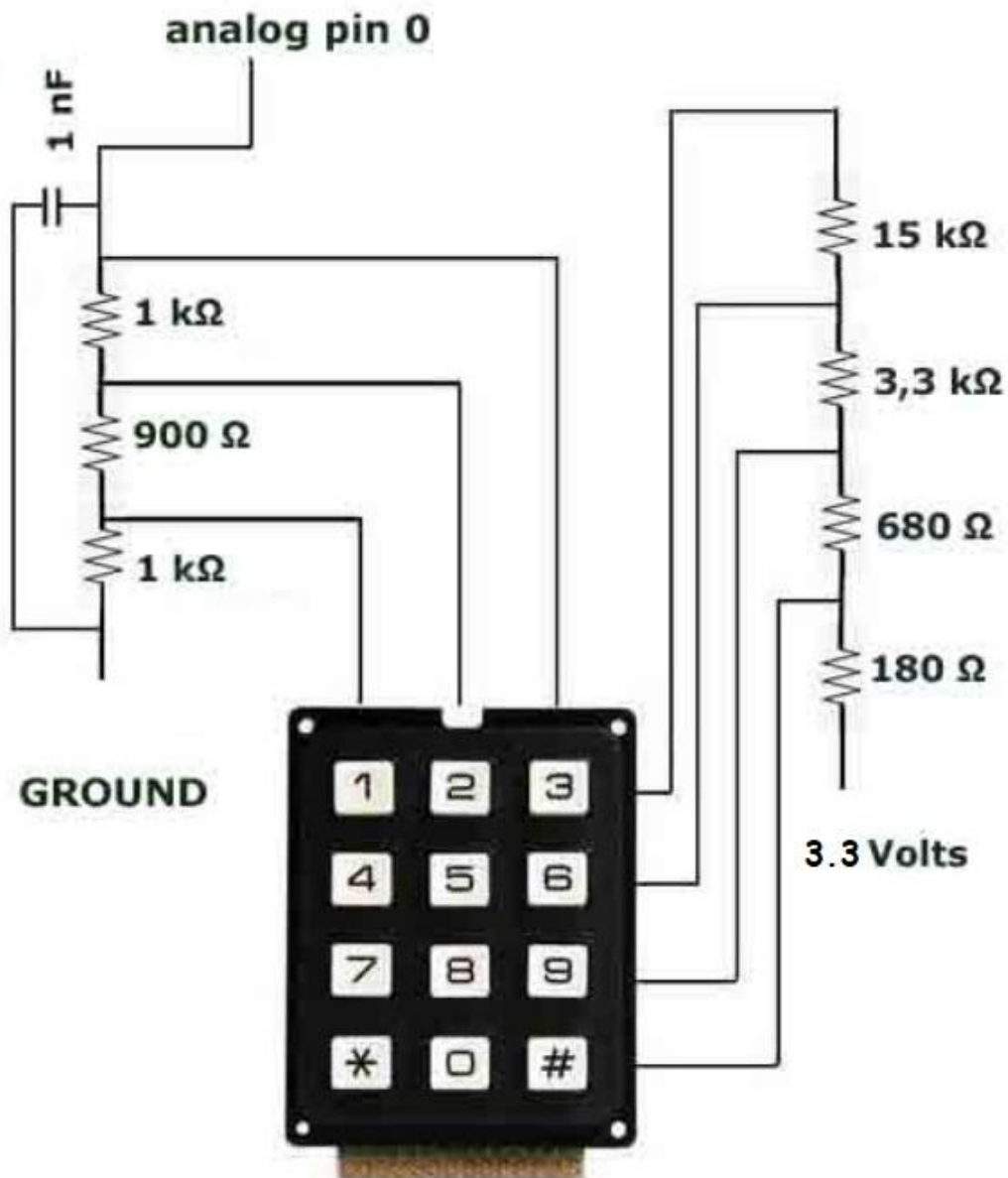
This interface was used to integrate the embedded controller with the physical alarm hardware while protecting the low-voltage control side.

4.3 Analog Keypad Interface

A standard 3×4 matrix keypad was selected for local user interaction. A conventional implementation would require seven microcontroller I/O connections. Because the available I/O resources were limited, an alternative analog interface was developed.

A resistor network was used to create a voltage divider in which each button produces a distinct output voltage. The resulting signal is connected to a single analog input of the controller, allowing all twelve keys to be distinguished using one ADC channel.

Figure 4. Analog voltage-divider interface for the 3×4 keypad.



The measured values ranged from approximately 0.16 V (ADC 48) for key 1 to 3.13 V (ADC 955) for key #. These measured values were subsequently used by the firmware to identify individual key presses.

This design reduced the keypad interface from seven required digital connections to a single analog input while preserving the complete 12-key interface.

4.4 Physical Prototype

The complete circuit was assembled on a prototype board and installed inside an electrical enclosure together with the controller, power circuitry, backup battery, keypad connections and alarm interfaces.

The physical implementation was used throughout development and testing, allowing the embedded firmware, power electronics, user interface, sensors and alarm outputs to be evaluated as an integrated system rather than as isolated subsystems.

5. Embedded Software Architecture

The embedded software was developed in C++ using the Arduino development environment. Rather than implementing the complete alarm logic directly within a single main program, the firmware was structured using an object-oriented approach in which the major physical and logical subsystems are represented by dedicated classes.

The main firmware is responsible for initialization, cloud synchronization and execution of the system control cycle, while the internal behavior of individual hardware components is handled through separate software abstractions. This approach separates device-specific functionality from the overall alarm logic and makes the interaction between the different subsystems easier to manage.

5.1 Firmware Components

The software architecture is organized around four principal classes:

microcontroller

Acts as the central coordinator of the local alarm system. It manages the overall operating logic and coordinates the interaction between sensors, sirens, keypad input and the system state.

sensor

Represents the connected alarm sensors and encapsulates their individual behavior. Sensor objects are responsible for monitoring their associated inputs and maintaining the information required by the main controller to determine how detected activity should affect the alarm system.

siren

Provides the software interface to the audible alarm outputs. This separates siren activation and control from the higher-level decision logic of the alarm system.

keypad

Handles the local 3×4 keypad interface. The class processes the analog input generated by the keypad voltage-divider circuit and provides the corresponding user-input functionality to the rest of the system.

The interaction between these components can be represented conceptually as:

Keypad + Sensors → microcontroller → Sirens / System State

At the same time, selected system information is exchanged with the Arduino IoT Cloud, allowing remote actions and cloud-side events to interact with the locally executed firmware.

5.2 Main Program Structure

During initialization, the firmware configures the required hardware interfaces and establishes the software objects used by the alarm system. Cloud-related properties are also initialized so that selected variables can be synchronized with the Arduino IoT Cloud.

During normal execution, the main program repeatedly services the local alarm logic and the cloud connection. The microcontroller class coordinates the local system behavior, while the supporting classes handle their respective hardware-related responsibilities.

This organization keeps the high-level system behavior separate from the implementation details of individual peripherals. For example, the central control logic does not need to reproduce the analog keypad decoding procedure every time user input is required; this functionality remains encapsulated within the keypad implementation.

5.3 Software Modularity

The use of separate classes reflects the physical organization of the system and provides a modular structure for the firmware. Sensors and output devices can be represented independently, while the central controller determines how their states affect the overall alarm behavior.

This separation was particularly useful because the project combines several different responsibilities within the same embedded platform: local user interaction, sensor supervision, alarm actuation, timing behavior and cloud communication.

The firmware architecture therefore provides the software foundation on which the alarm-control logic is implemented. The transitions between the different operating conditions, detection stages and alarm responses are discussed separately in the following section.

6. Alarm Control and State Logic

The operation of the alarm system is controlled locally by the embedded firmware. The controller continuously evaluates the current operating condition, sensor inputs, timing parameters and user interaction in order to determine the appropriate system response.

The control logic was designed so that sensor activity does not necessarily result in immediate alarm activation. Instead, the response depends on the current system condition and the behavior assigned to the corresponding sensor. This allows the same controller to coordinate normal monitoring, sensor detection, remote notification, pre-alarm behavior and physical alarm activation.

The system supports both local and remote interaction. Core alarm functionality is executed locally by the embedded controller, ensuring that the system can continue to operate independently of the remote interface. At the same time, cloud connectivity provides a convenient method of remotely controlling and monitoring the system and receiving notifications when relevant events occur.

6.1 System Activation and Sensor Warm-Up

When the alarm system is activated, the controller prepares the required sensors for operation. Certain sensing devices require a stabilization or warm-up period before their output can be considered reliable. The firmware therefore incorporates timing behavior that allows the required initialization period to complete before normal monitoring begins.

During this stage, the controller manages the transition from system activation to active monitoring without interpreting temporary sensor behavior during initialization as a valid alarm event.

Once the required preparation has been completed, the relevant sensors become part of the active monitoring process and their inputs are continuously evaluated by the controller.

This process is handled locally by the embedded firmware and does not depend on the availability of the cloud platform or remote interface.

6.2 Sensor Monitoring and Event Evaluation

During normal operation, the controller monitors the connected sensor inputs and evaluates detected activity according to the configuration of the corresponding sensor and the current operating condition of the system.

A sensor event is first registered by the firmware before the appropriate response is determined. This separation between detection and response allows different sensors or events to produce different system behavior.

An event may therefore be used primarily for monitoring or remote notification without immediately activating the main alarm, while another event may participate directly in the alarm sequence. This provides greater flexibility than treating every detected sensor transition as an identical alarm condition.

The sensor-management logic remains separated from the overall coordination of the system through the firmware architecture described in the previous section. Individual sensor objects handle their associated behavior, while the main controller coordinates the resulting system response.

6.3 Pre-Alarm, Alarm Response and Timing

When a detected condition requires an alarm response, the controller manages the progression toward physical alarm activation.

The implementation includes a pre-alarm stage, providing an intermediate audible warning and a period during which the system can respond to user interaction before the main alarm output is activated. This creates a controlled transition between detection and full alarm activation rather than treating the two as a single instantaneous event.

If the triggering condition progresses to a full alarm event, the main siren is activated locally through the corresponding output-control circuitry. Timing logic is used to manage the alarm sequence and its duration.

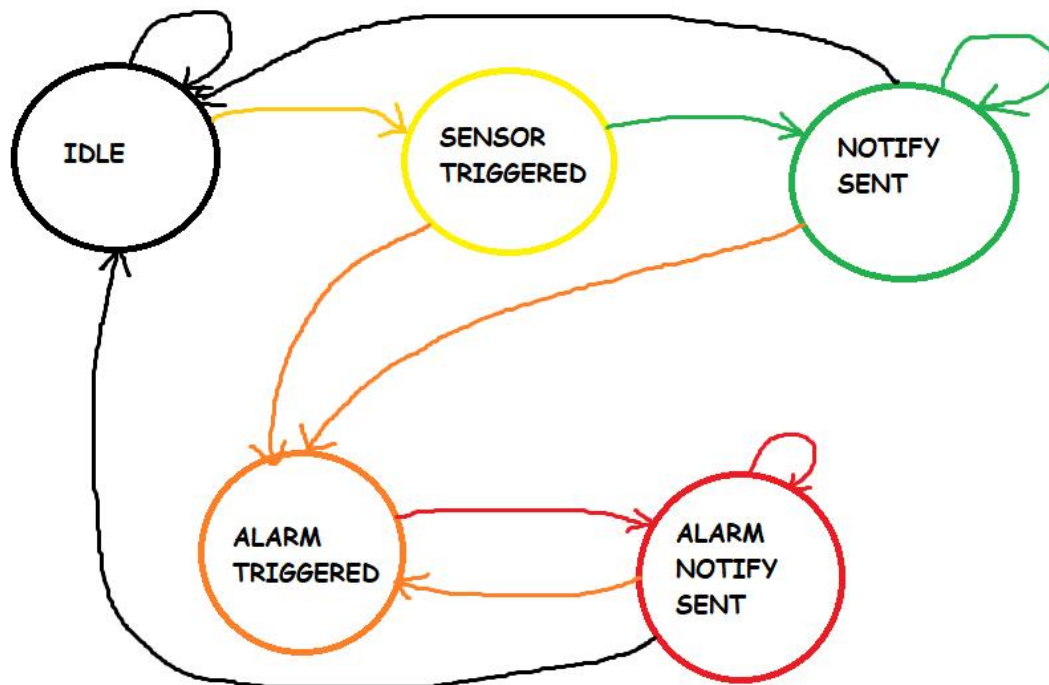
The local keypad remains available as part of the system interaction, allowing the embedded controller to respond to authorized user input according to the current operating condition.

Relevant changes in the system condition can simultaneously be synchronized with the cloud platform. This allows sensor events, pre-alarm conditions and alarm events to be reflected remotely and, where configured, to generate corresponding notifications.

6.4 State Machine

The high-level behavioral logic of the alarm system is represented by the state machine shown below. The diagram describes the principal states associated with sensor detection, notification and alarm activation.

Figure 5. State-machine representation of the alarm control logic.



The IDLE state represents the condition in which no active sensor or alarm event is currently being processed. When relevant sensor activity is detected, the system transitions to SENSOR TRIGGERED, indicating that an input event has been registered and must be handled by the control logic.

From this point, the subsequent transition depends on the required response. The system can progress to NOTIFY SENT, representing the reporting of a detected sensor event, or toward ALARM TRIGGERED when the detected condition requires an alarm response.

The NOTIFY SENT state separates the reporting of sensor activity from physical alarm activation. This allows detected events to be communicated remotely without requiring every notification-producing event to result in activation of the main siren.

When an alarm condition is reached, the system enters ALARM TRIGGERED, representing the physical alarm response. Remote reporting associated specifically with the alarm condition is represented by the ALARM NOTIFY SENT state.

The transitions shown in the state machine allow the control logic to progress between these conditions as events are processed and ultimately return to IDLE when the corresponding sequence has been completed.

The diagram intentionally provides a high-level representation of system behavior. Internal operations such as sensor warm-up, timing, keypad processing and individual hardware actions are handled by the firmware and are therefore not represented as independent states.

6.5 Local and Remote Control

The alarm system was designed to support both local and remote control, with each method serving an important practical purpose.

Local operation is handled directly by the embedded controller and remains available independently of the remote interface. The physical keypad provides a means of interacting with the alarm system at the installation site, while sensor evaluation, timing, state management and physical alarm-output control are executed by the embedded firmware.

Maintaining this local functionality is particularly important for reliability and accessibility. The fundamental operation of a security system should not depend exclusively on the availability of an Internet connection, cloud service or mobile device. Loss of connectivity therefore does not remove the underlying local alarm functionality.

At the same time, remote control is a significant feature of the system rather than simply an additional monitoring mechanism. Through the Arduino IoT Cloud interface, supported system functions can be controlled remotely from a mobile device. This provides a convenient method of everyday interaction without requiring physical access to the alarm panel and keypad for every operation.

The remote interface also provides access to system status information, allowing the user to monitor the current condition of the alarm system while away from the installation. In addition, relevant sensor and alarm events can be propagated through the cloud platform to generate remote notifications.

The two control methods therefore complement each other rather than compete with one another. Local embedded control provides autonomous operation and independence from network connectivity, while the IoT interface provides convenient remote control, monitoring and event notification.

This architecture preserves the reliability expected from a locally operated alarm system while adding the accessibility and convenience provided by an Internet-connected platform. The implementation of the cloud interface, synchronized variables, remote controls and notification functionality is described in the following section.

7. Arduino IoT Cloud Integration

The IoT functionality of the system was implemented using the Arduino IoT Cloud, providing a communication layer between the embedded alarm controller and the remote user interface. The platform was used to synchronize selected system variables, provide remote controls and status information, and generate notifications in response to relevant events.

The cloud functionality was designed as an extension of the locally operating alarm system. The embedded controller remains responsible for the fundamental alarm logic, while the cloud platform provides convenient remote access to selected functions and system information.

7.1 Cloud Communication

The ESP8266 controller establishes an Internet connection and communicates with the Arduino IoT Cloud through the cloud configuration integrated into the firmware.

Selected variables are synchronized between the embedded system and the cloud platform. Depending on their purpose, these variables can represent system states, sensor-related information or remotely controllable parameters.

This bidirectional communication allows changes originating from the embedded system to be reflected in the remote interface while also allowing supported user commands from the cloud interface to affect the operation of the alarm system.

The resulting communication path can be summarized as:

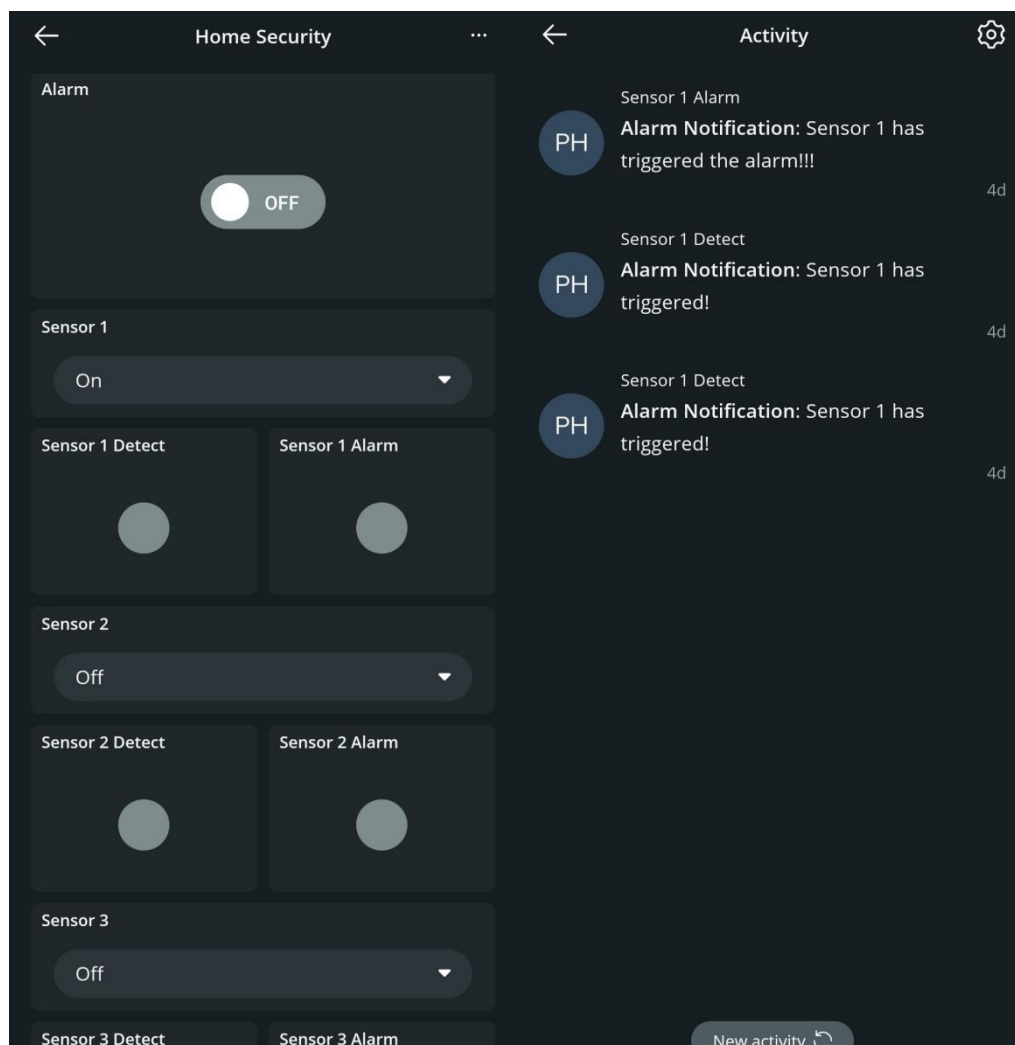
Alarm Hardware ↔ Embedded Firmware ↔ Arduino IoT Cloud ↔ Remote User Interface

7.2 Remote User Interface

The remote interface was created using the configurable elements provided by the Arduino IoT Cloud platform. Rather than developing a separate mobile application, the available dashboard components were configured and adapted to provide the controls and indicators required by the alarm system.

Through this interface, the user can remotely access supported alarm functions and monitor relevant information from a mobile device. This makes routine interaction considerably more convenient when physical access to the alarm keypad is unnecessary or unavailable.

Figure 6.Arduino IoT Cloud mobile interface: (a) remote alarm control and sensor monitoring dashboard; (b) cloud-generated sensor detection and alarm notifications.



The remote interface complements the physical keypad rather than replacing it. Both methods provide access to the system, while the local embedded implementation ensures that fundamental alarm functionality remains available independently of the cloud connection.

7.3 Remote Monitoring and Control

Cloud synchronization provides both monitoring and control capabilities.

System information generated locally can be reflected remotely, allowing the user to observe relevant operating conditions without being physically present at the installation. At the same time, cloud-connected controls allow supported commands to be transmitted back to the embedded controller.

This bidirectional architecture is an important aspect of the implementation: the cloud connection is not limited to passive visualization of sensor data. It forms an additional user interface through which the alarm system can be actively operated.

The remote interface therefore provides three principal IoT capabilities:

Remote control of supported alarm functions.

Remote monitoring of system information and operating conditions.

Remote notification following relevant sensor and alarm events.

7.4 Cloud-Based Notifications

The Arduino IoT Cloud was also used to implement remote event notification. Relevant changes in synchronized variables can be associated with cloud-side triggers, allowing predefined events detected by the embedded system to generate notifications.

The notification mechanism complements the local audible response. A sensor event can therefore be communicated to the user remotely even when the user is away from the protected location, while alarm-related events can similarly be reflected through the cloud platform.

The distinction between sensor-event notification and alarm notification is also represented in the high-level state machine described in the previous section through the NOTIFY SENT and ALARM NOTIFY SENT states.

7.5 Role of the Cloud Platform

The use of the Arduino IoT Cloud reduced the need to implement a dedicated communication backend and mobile application while still allowing the prototype to demonstrate practical IoT functionality.

The platform provides the communication infrastructure and configurable interface elements, while the project-specific behavior—including the alarm logic, hardware integration, sensor processing and interaction with the synchronized variables—is implemented within the embedded system.

This approach was appropriate for the objectives of the prototype, allowing development effort to remain focused on the integration of the physical alarm system with embedded and IoT functionality while still providing a practical and accessible remote interface.

8. Testing and Evaluation

The completed prototype was tested as an integrated system under real operating conditions. The evaluation included the interaction between the embedded controller, connected sensors, keypad, alarm outputs and Arduino IoT Cloud functionality.

Local operation was verified through sensor activation, keypad input and alarm-output control. The controller was able to manage the different functions of the system using a single NodeMCU, including sensor monitoring, siren control, keypad processing, cloud communication and the initiation of remote notifications.

The cloud-connected functionality was also evaluated through the configured remote interface. Changes in system and sensor conditions were reflected through the Arduino IoT Cloud, while the corresponding remote controls allowed supported system functions to be operated from the mobile interface. Cloud triggers were used to generate email and push notifications when the associated cloud variables reached predefined conditions.

Testing of the complete prototype confirmed the practical operation of the system and demonstrated stable interaction between its local and cloud-connected components. The implementation therefore demonstrated that the required alarm, monitoring, remote-control and notification functionality could be handled using a single embedded controller and a relatively small amount of supporting hardware.

9. Cost Analysis

Low implementation cost was one of the main considerations of the project. Based on the component prices considered during development, the complete prototype hardware was estimated at approximately €40–50.

The principal costs included the NodeMCU controller, prototype board and electronic components, keypad, sensors, sirens, backup battery and electrical enclosure. The individual components were inexpensive, with the sensors and enclosure representing some of the more significant portions of the overall hardware cost.

At the time of the original project, use of the Arduino IoT Cloud functionality required for triggers was estimated at approximately \$6 per month. This subscription was not considered essential to the underlying local alarm operation and could potentially be replaced by alternative services if recurring cloud costs were to be avoided.

For larger-scale implementation, the original analysis identified several opportunities for further cost reduction, including the development of a custom PCB, the use of a smaller standardized enclosure and bulk purchasing or substitution of sensors with lower-cost alternatives.

10. Limitations and Future Development

The implemented system was developed as a functional prototype and therefore leaves several opportunities for further engineering development. The prototype-board construction could be replaced by a dedicated printed circuit board (PCB), reducing physical size, simplifying wiring and providing a more suitable implementation for repeated

production. A smaller purpose-designed enclosure could similarly improve the physical integration of the system.

Several functional extensions were also identified during the project. A future version could incorporate a camera for real-time visual monitoring, allowing alarm events to be supplemented with visual information. More advanced processing could introduce machine-learning techniques for event classification or pattern recognition, while voice-control functionality could provide an additional method of user interaction.

Further development could also target installations where conventional network or power infrastructure is unavailable. The use of solar charging could increase system autonomy, while alternative communication technologies such as LoRa could extend the concept to remote locations where conventional Internet connectivity is impractical.

These extensions are not required for the fundamental operation demonstrated by the prototype, but represent possible directions through which the architecture could evolve into a more capable and application-specific security platform.

11. Conclusion

This project demonstrated the design and implementation of a complete low-cost IoT alarm system combining embedded control, conventional alarm hardware and cloud-based functionality.

The resulting prototype integrates sensor monitoring, local keypad interaction, audible alarm outputs, remote control, live monitoring and automated notifications within a single system. The use of the Arduino IoT Cloud provided the required remote functionality without requiring the development and maintenance of a dedicated local server or custom backend infrastructure.

An important characteristic of the implementation is that the IoT functionality complements rather than replaces the embedded alarm system. Core security functions are executed locally by the controller, while cloud connectivity provides an additional and convenient interface for remote control, monitoring and event notification.

The complete functionality was implemented around a single NodeMCU controller, which manages the keypad, sensors, siren outputs, cloud communication and notification-related logic. Testing of the physical prototype demonstrated the practical operation and integration of these functions under real operating conditions.

With an estimated hardware cost below €50 at the time of development, the project also demonstrated that practical IoT security functionality can be implemented using inexpensive and readily available components.

Overall, the project provides a functional example of the integration of embedded systems, electronic hardware, IoT communication and cloud services into a unified security platform, while retaining a relatively simple, modular and low-cost implementation.

12. Related Academic Publication

The alarm system presented in this report was also used as a practical case study in a peer-reviewed conference publication investigating the use of IoT Cloud services for simplifying remote control and monitoring of microcontroller-based automation systems.

The publication examines an IoT Cloud-based architecture as an alternative to traditional approaches requiring a continuously operating local server. The home-security system developed in this project is presented as the practical implementation used to evaluate this approach, including its NodeMCU-based architecture, voltage regulation, electrical isolation, analog keypad interface, sensor state management, remote dashboard and cloud notification functionality.

The broader study concludes that IoT Cloud platforms can reduce infrastructure requirements and implementation complexity while providing remote access, customization and scalability. The alarm prototype therefore served not only as a standalone embedded and IoT project, but also as an experimental case study supporting the broader investigation of cloud-based control architectures for automation systems.

Publication:

F. Kladouchas and N. N. Karanikolas, **“Simplify the creation of remote controls and monitoring interfaces for microcontrollers and automation systems using IoT Cloud services”** *RTA-CSIT 2025: 6th International Conference on Recent Trends and Applications in Computer Science*, Tirana, Albania, May 22–24, 2025, CEUR Workshop Proceedings, Vol. 4044, 2025.

[View the published paper — CEUR Workshop Proceedings](#)